RELATÓRIO DE CONCLUSÃO

Instrumentar erros e latência

Projeto: Seminário Técnico — Grafana

Curso: Sistemas de Informação — UniSales

Ambiente validado: Node.js + Prometheus + Grafana + Docker Compose

Status: CONCLUÍDA

Data de validação: 19/10/2026

1. Objetivo da entrega

A segunda tarefa teve como objetivo permitir a identificação de falhas HTTP e medir o tempo de resposta da API. Esses dados são necessários para avaliar confiabilidade e desempenho da aplicação durante a demonstração.

2. Instrumentação de latência

Foi criado o histograma api_http_request_duration_seconds. O middleware registra o instante inicial com process.hrtime.bigint(), calcula a duração ao final da resposta e envia o valor ao histograma com as labels method, route e status_code.

```
api_http_request_duration_seconds
Buckets: 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5 e 5 segundos
```

O histograma permite calcular percentis de latência. No dashboard já provisionado, são calculados p50 e p95 por meio de histogram_quantile sobre os buckets coletados pelo Prometheus.

```
histogram_quantile(0.95, sum by (le)
(rate(api_http_request_duration_seconds_bucket{job="api"}[5m])))
histogram_quantile(0.50, sum by (le)
(rate(api_http_request_duration_seconds_bucket{job="api"}[5m])))
```

3. Instrumentação de erros

O planejamento original previa um contador separado chamado http_errors_total. A implementação final adotou uma abordagem mais enxuta: erros são identificados pela label status_code da própria api_http_requests_total. Assim, respostas 404, 500 ou qualquer outra classe HTTP são registradas sem duplicar eventos em uma segunda métrica.

```
api_http_requests_total{status_code="500"}
api_http_requests_total{status_code=~"5.."}
```

4. Evidência de funcionamento

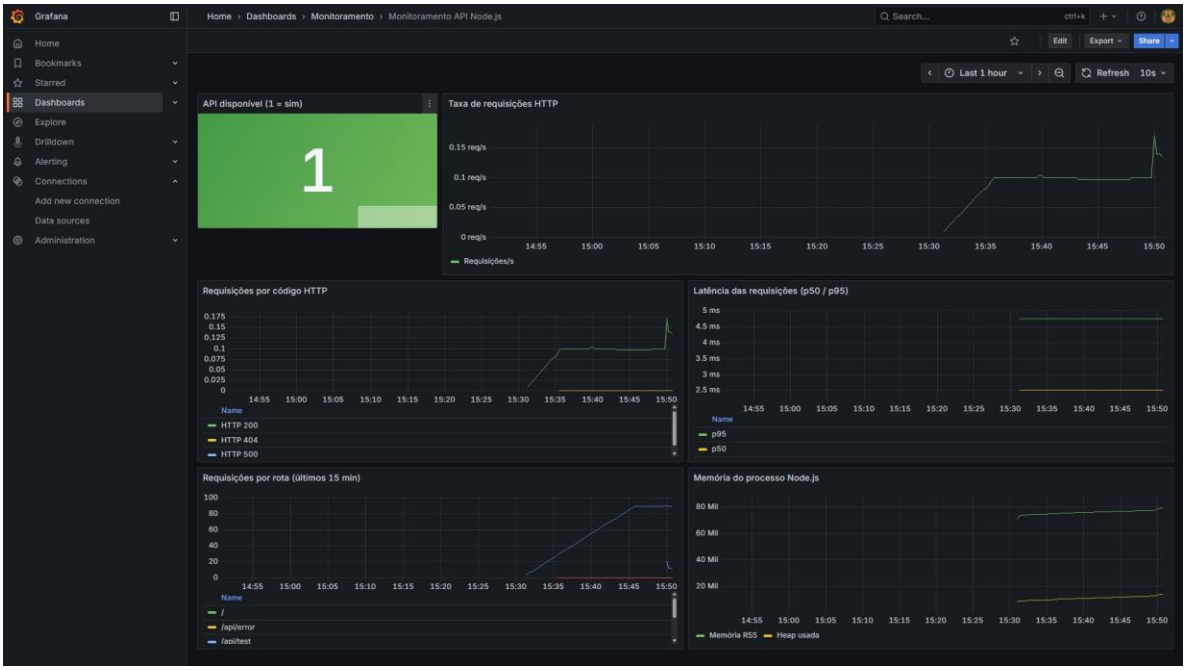


Figura 1 – Dashboard mostrando HTTP 500 e os painéis de latência p50/p95 após o teste.
Fonte: evidência produzida durante a validação prática do projeto (2026).

Após chamadas ao endpoint /api/error, o painel de códigos HTTP passou a apresentar a série HTTP 500. Ao mesmo tempo, o painel de latência continuou exibindo p50 e p95, comprovando que as durações das requisições estavam sendo processadas.

5. Alteração em relação ao planejamento original

Planejamento original	Implementação validada
Criar http_errors_total e http_request_duration_seconds.	Erros foram representados por api_http_requests_total filtrada pela label status_code; a duração foi implementada como api_http_request_duration_seconds, do tipo Histogram.

Impacto da mudança: A mudança elimina redundância no contador de erros e mantém toda a capacidade analítica necessária. A métrica de latência ganhou prefixo api_ para padronização.

6. Critérios de conclusão

Critério	Resultado	Status
Erros HTTP	Respostas 4xx/5xx registradas em status_code.	**Concluído**
Erro 500	Endpoint /api/error validado e série HTTP 500 observada.	**Concluído**
Latência	api_http_request_duration_seconds disponível.	**Concluído**
Percentis	p50 e p95 calculados no Grafana.	**Concluído**
Rastreabilidade	Método, rota e status mantidos também na latência.	**Concluído**

7. Conclusão

A API registra falhas e duração das chamadas de forma suficiente para análise operacional. A implementação adotada difere do nome de algumas métricas originalmente planejadas, mas atende integralmente ao objetivo técnico da tarefa.